

Сравнение производительности форматов данных: CSV, JSONL, Parquet

Вводные данные

Датасет: [tabularisai/oak](#) — синтетический датасет для тренировки LLMок.

- **Количество записей:** 1,055,633
- **Количество колонок:** 13
- **Размер (CSV):** 3.4 GB
- **Размер (JSONL):** 3.64 GB
- **Размер (Parquet):** 1.06 GB

Инструменты:

- **Criterion:** фреймворк для бенчмаркинга (на каждом датасете смотрят по 10 итераций в среднем).
- **Polars:** библиотека для обработки данных.

Бенчмарк 1: Базовый запрос

К трем форматам применяется следующий query:

```
fn bench_func(lf: LazyFrame) {
  lf.select([col("Prompt").str().len_chars(), col("Category")])
    .filter(col("Category").eq(lit("Music")))
    .select([col("Prompt").sum()])
    .collect()
    .unwrap();
}
```

Результаты:

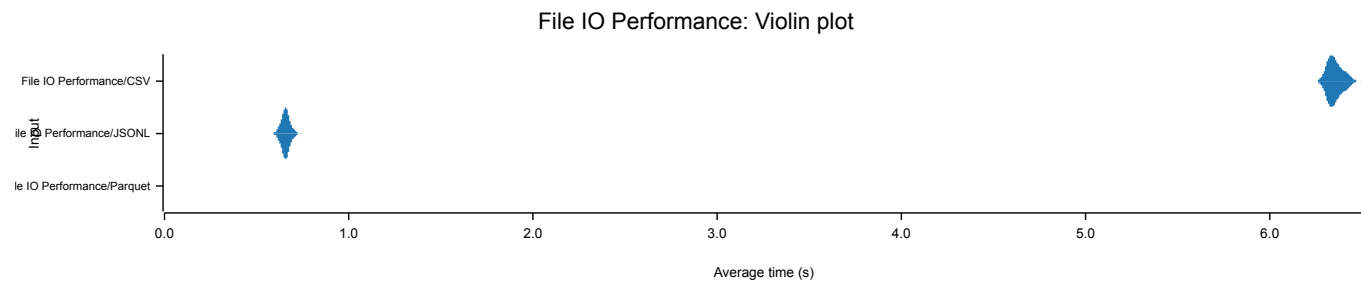
Метрика	CSV	JSONL	PARQUET
Максимальное	6.43 s	702.64 ms	58.57 ms
Минимальное	6.32 s	625.20 ms	57.98 ms
Среднее	6.36 s	661.18 ms	58.22 ms
Ratio с CSV	1.0	~0.10 (10x faster)	~0.009 (110x faster)

Сравнение с однопоточным режимом (Single-threaded):

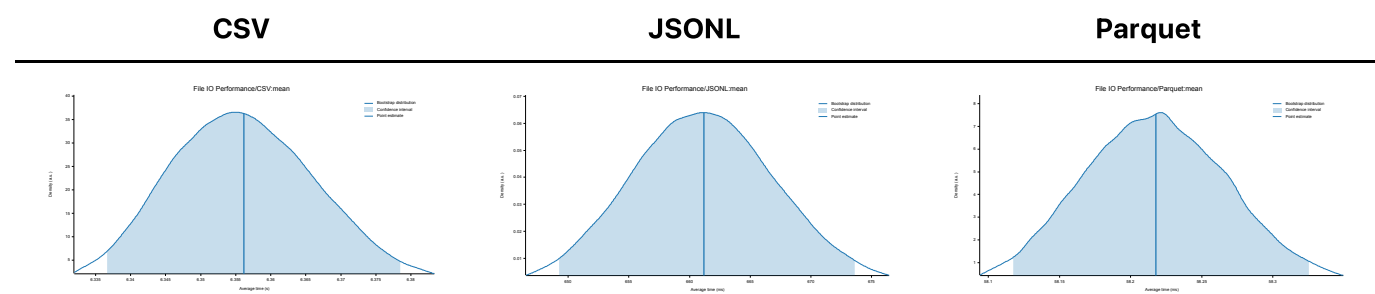
- **CSV:** 8.28 s (Single) vs 6.36 s (Multi). Многопоточность ускоряет на ~30%.
- **JSONL:** 2.87 (Single) vs 661.18 ms (Multi). Многопоточность ускоряет на ~400%.
- **Parquet:** 85.48 ms (Single) vs 58.22 ms (Multi). Многопоточность ускоряет на ~40%.

•

График:



Средние:



Бенчмарк 2: min-max Performance

Простой запрос, вычисляющий min, max по одной колонке. Parquet вытаскивает из метадаты

```
fn bench_func(lf: LazyFrame) {  
  
  lf.clone().select([col("Prompt_token_length").min()]).collect().unwrap();  
  lf.select([col("Prompt_token_length").max()]).collect().unwrap();  
}
```

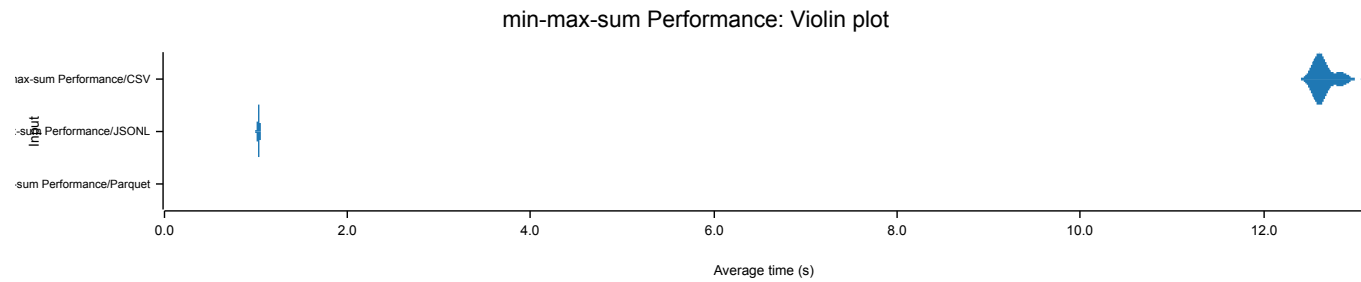
Результаты:

Метрика	CSV	JSONL	PARQUET
Максимальное	12.85 s	1.06 s	1.88 ms
Минимальное	12.55 s	1.02 s	1.83 ms
Среднее	12.65 s	1.03 s	1.84 ms
Ratio с CSV	1.0	~0.08 (12x faster)	~0.00015 (6800x faster)

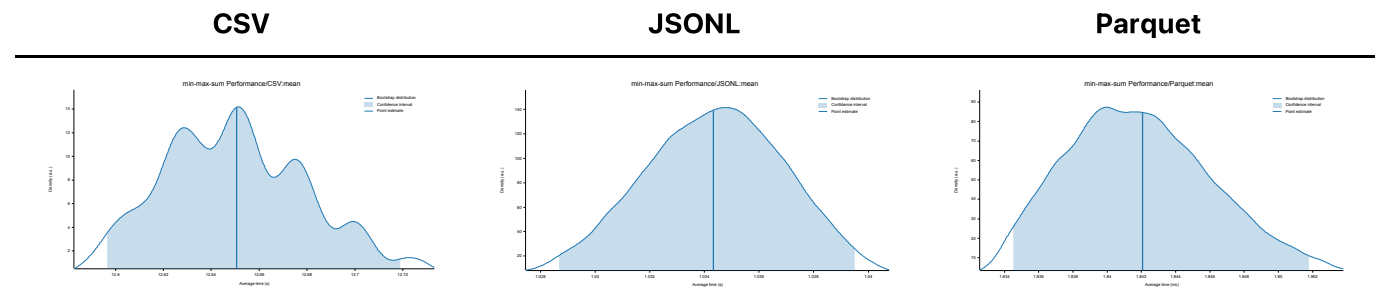
Сравнение с однопоточным режимом (Single-threaded):

- **CSV:** 15.495 s (Single) vs 12.65 s (Multi). Многопоточность ускоряет на ~22%.
- **JSONL:** 6.2451 s (Single) vs 1.03 s (Multi). Многопоточность ускоряет примерно в 6 раз.
- **Parquet:** 3.7356 ms (Single) vs 1.84 ms (Multi). Многопоточность ускоряет примерно в 2 раза.

График:



Средние:



Бенчмарк 3: Аналитический запрос (Group By + Agg)

Запрос выполняет фильтрацию, группировку по двум полям и агрегацию (среднее, максимум, количество).

```
fn bench_func(lf: LazyFrame) {  
  lf.filter(col("Selected_score").eq(lit("first-class")))  
    .group_by([col("Category"), col("Prompt_model")])  
    .agg([  
      col("Prompt_token_length").mean().alias("avg_input_tokens"),  
      col("Response_token_length")  
        .max()  
        .alias("max_output_tokens"),  
      len().alias("count"),  
    ])  
    .collect()  
    .unwrap();  
}
```

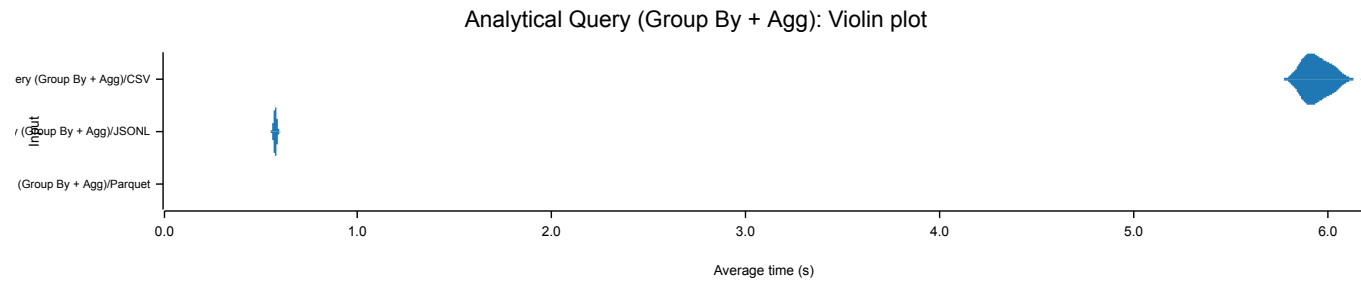
Результаты:

Метрика	CSV	JSONL	PARQUET
Максимальное	6.05 s	587.22 ms	2.96 ms
Минимальное	5.87 s	563.71 ms	2.65 ms
Среднее	5.94 s	574.90 ms	2.76 ms
Ratio с CSV	1.0	~0.1 (10x faster)	~0.00046 (2150x faster)

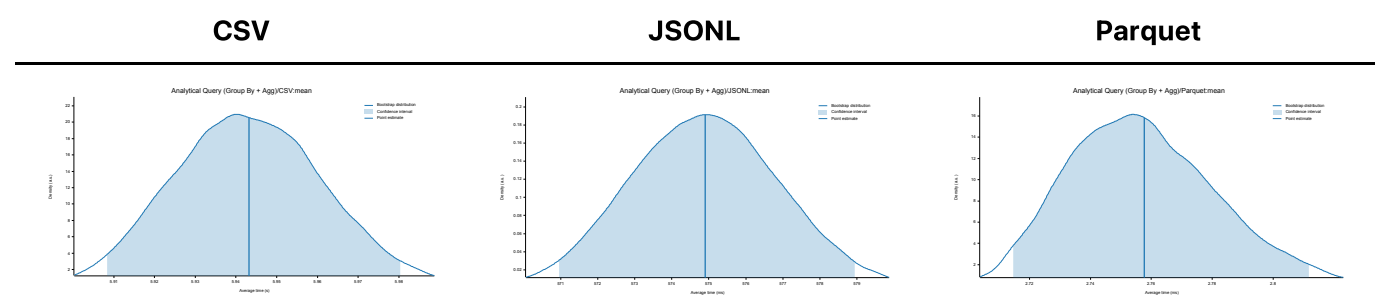
Сравнение с однопоточным режимом (Single-threaded):

- **CSV:** 8.28 s (Single) vs 5.94 s (Multi). Многопоточность ускоряет на ~39%.
- **JSONL:** 574.90 ms (Multi) - данные для однопоточного режима отсутствуют.
- **Parquet:** 6.66 ms (Single) vs 2.76 ms (Multi). Многопоточность ускоряет примерно в 2.4 раза.

График:



Средние:



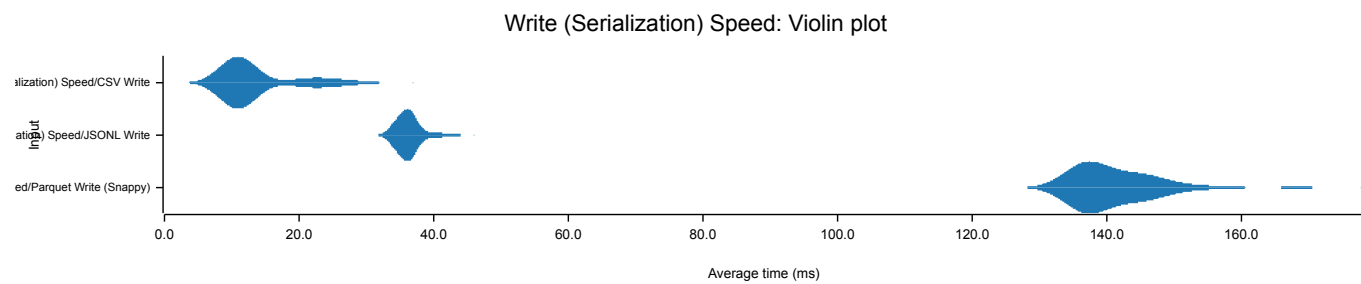
Бенчмарк 4: Скорость записи

Тест замеряет время записи 10,000 строк в файл.

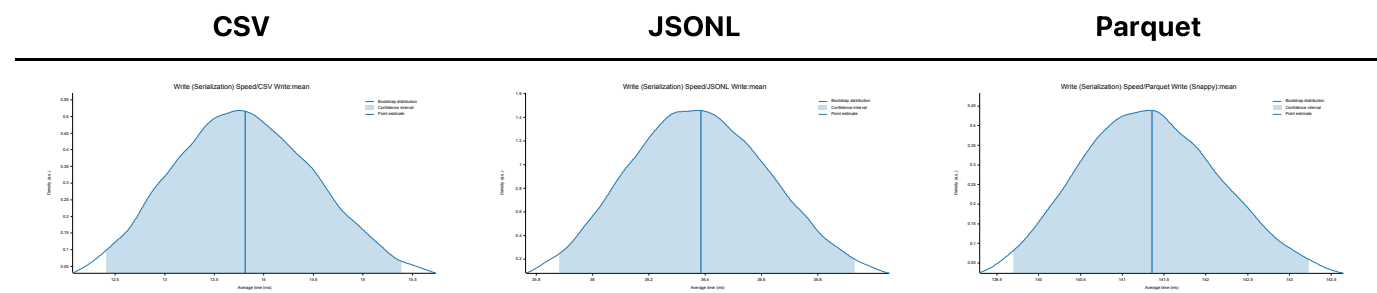
Результаты:

Метрика	CSV	JSONL	PARQUET (Snappy)
Максимальное	45.53 ms	46.71 ms	166.91 ms
Минимальное	9.69 ms	35.63 ms	134.41 ms
Среднее	15.70 ms	37.75 ms	141.03 ms
Ratio с CSV	1.0	~2.4 (2.4x slower)	~9.0 (9x slower)

График:



Средние:



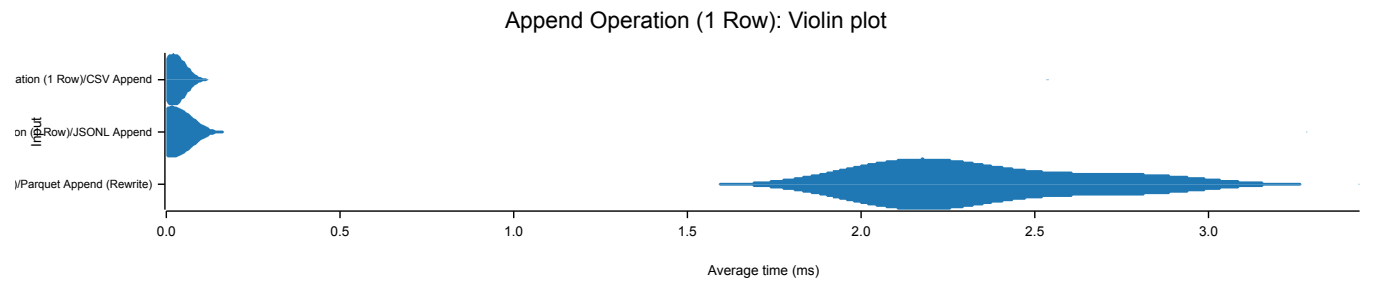
Бенчмарк 5: Добавление строки (Append)

Тест замеряет время добавления одной новой записи в конец файла. Для CSV и JSONL это простая операция дозаписи (append). Для Parquet это требует чтения всего файла, добавления строки и полной перезаписи (так как Parquet иммутабелен).

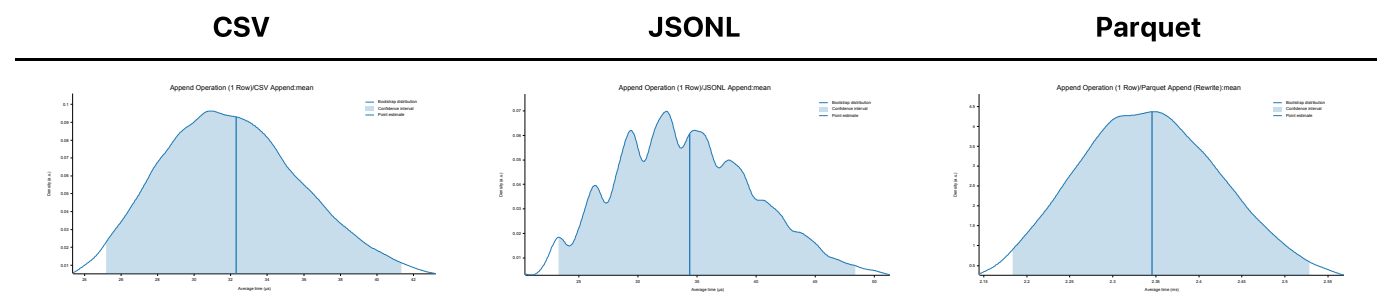
Результаты:

Метрика	CSV	JSONL	PARQUET (Rewrite)
Максимальное	2.11 ms	2.02 ms	4.58 ms
Минимальное	15.71 μs	17.78 μs	2.07 ms
Среднее	33.44 μs	33.52 μs	2.73 ms
Ratio с CSV	1.0	~1.0 (Same speed)	~81.6 (80x slower)

График:



Средние:



Бенчмарк 6: Скорость сжатия (Compression Speed)

Тест замеряет время записи с использованием разных алгоритмов сжатия для Parquet. Сравниваем CSV (без сжатия), Parquet Snappy (стандарт) и Parquet Zstd (сильное сжатие).

Размер файлов

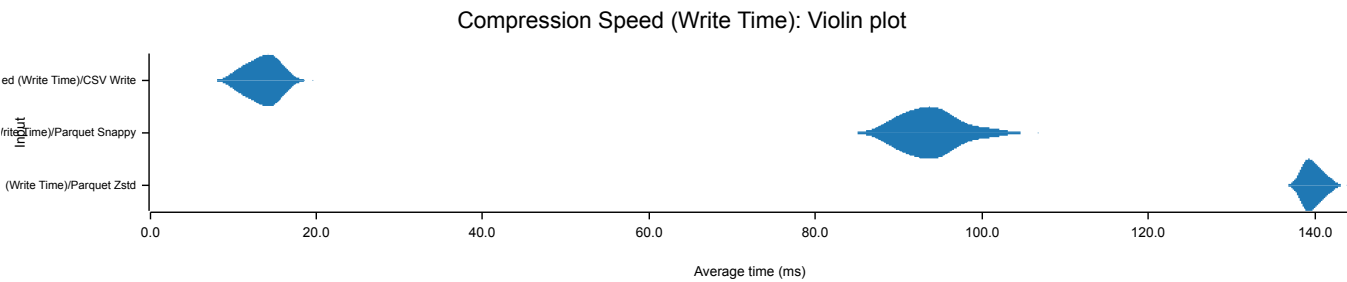
Format	Size	Ratio (vs CSV)
CSV (Raw)	33.59 MB	100.0%
JSONL	35.97 MB	107.1%
Parquet (Snappy)	19.33 MB	57.5%
Parquet (Zstd)	11.26 MB	33.5%

Время записи

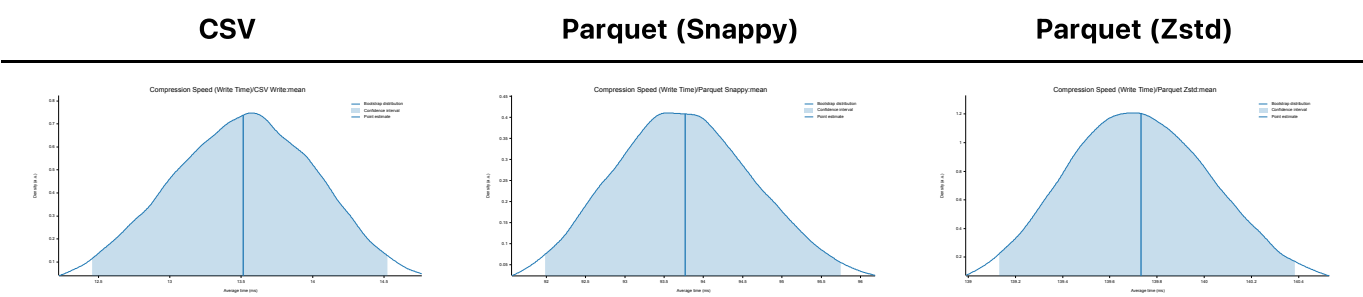
Результаты:

Метрика	CSV	Parquet (Snappy)	Parquet (Zstd)
Максимальное	21.76 ms	90.30 ms	141.86 ms
Минимальное	10.03 ms	88.16 ms	138.12 ms
Среднее	15.43 ms	88.97 ms	139.23 ms
Ratio с CSV	1.0	~5.8 (6x slower)	~9.0 (9x slower)

График:



Средние:



Выводы

Характеристика	CSV	JSONL	Parquet
Чтение (Read)	Медленно (парсинг текста)	Медленно (парсинг JSON)	Очень быстро (бинарный, колоночный)

Характеристика	CSV	JSONL	Parquet
Запись (Write)	Очень быстро (простой текст)	Быстро	Медленно (кодирование + сжатие)
Добавление (Append)	Мгновенно (O(1))	Мгновенно (O(1))	Очень медленно (требуется перезапись файла)
Размер (Size)	Большой (текст)	Самый большой (дублирование ключей)	Компактный (сжатие, словари)
Сложные запросы	Медленно (скан всего файла)	Медленно (скан всего файла)	Очень быстро (push-down predicates, projection)
Использование	Обмен данными, логи, простые таблицы	Логи, неструктурированные данные, веб-API	Аналитика, большие данные, ML-датасеты